

Tap_pa_rpa

INTEGRANTES:

LUIS GUILLERMO SOCARA POLO

ANIBAL ANTONIO CUADRADO RENGIFO

ANTONY MARTINEZ CANTERO

CORREO INSTITUCIONAL:

amartinezcantero49@correo.unicordoba.edu.co

lsoracapolo@correo.unicordoba.edu.co

acuadradorengifo75@correo.unicordoba.edu.co

Descripción General

El sistema de cuestionario es una aplicación web interactiva desarrollada con Nuxt 3 y Vuetify que permite a los usuarios responder preguntas de opción múltiple sobre Inteligencia Artificial, jugar con un modelo de agentes inteligentes y aprender sobre estos. El sistema evalúa automáticamente las respuestas y proporciona retroalimentación detallada.

Características Principales

- ✓ Cuestionario interactivo con navegación libre
- ✓ Evaluación automática de respuestas
- ✓ Barra de progreso visual
- ✓ Resultados detallados con revisión pregunta por pregunta
- ✓ Persistencia de datos en localStorage
- ✓ Interfaz responsive y moderna
- ✓ Opción de reintentar el cuestionario
- ✓ Agente Inteligente en ejecución
- ✓ Edición De Usuarios
- ✓ Proceso de Seguimiento

Tecnologías Utilizadas

Framework: Nuxt 3

UI Framework: Vuetify 3

State Management: Pinia

Persistencia: localStorage

Lenguaje: TypeScript/JavaScript

Componentes Principales

1. Hero Section

Propósito: Presentación principal del OVA

Título llamativo con gradiente de fondo

Descripción del contenido

Navegación fluida al contenido principal

2. Progress Section

Propósito: Seguimiento del progreso del usuario

Barra de progreso visual

Contador de módulos completados

Motivación visual del avance

3. Objectives Section

Propósito: Definir metas de aprendizaje

Tarjetas interactivas para cada objetivo

Sistema de marcado de completado

Iconografía representativa

4. Modules Section

Propósito: Contenido educativo estructurado

Paneles expansibles con contenido detallado

Indicadores de estado de progreso

Lista de contenidos por módulo

5. Activities Section

Propósito: Actividades prácticas y evaluativas

Tarjetas de actividades con metadata

Sistema de favoritos

Control de disponibilidad por prerrequisitos

6. Evaluations Section

Propósito: Sistema de logros y certificaciones

Barras de progreso para cada logro

Indicadores visuales de completion

Sistema de reconocimientos

Estructura de Datos

Objetivos de Aprendizaje

```
javascript{  
  id: Number,      // Identificador único  
  titulo: String,   // Nombre del objetivo  
  descripcion: String, // Descripción detallada  
  icono: String,    // Icono de Material Design  
  completado: Boolean // Estado de completado  
}
```

Módulos de Contenido

```
javascript{  
  id: Number,      // Identificador único  
  titulo: String,   // Título del módulo  
  descripcion: String, // Descripción del módulo  
  duracion: String, // Tiempo estimado  
  icono: String,    // Icono representativo  
  completado: Boolean, // Estado de completado  
  contenidos: Array // Lista de temas incluidos  
}
```

Actividades de Aprendizaje

```
javascript{  
  id: Number,      // Identificador único  
  titulo: String,   // Nombre de la actividad  
  descripcion: String, // Descripción de la actividad  
  tipo: String,     // 'practica' | 'evaluacion'  
  duracion: String, // Tiempo estimado  
  dificultad: Number, // Escala 1-5  
  disponible: Boolean, // Si está habilitada  
  favorito: Boolean, // Si está marcada como favorita  
  imagen: String,   // URL de imagen representativa  
  ruta: String      // Ruta de navegación  
}
```

Logros y Certificaciones

```
javascript{  
  id: Number,      // Identificador único  
  titulo: String,   // Nombre del logro
```

```
descripcion: String, // Descripción del logro
progreso: Number,    // Porcentaje 0-100
obtenido: Boolean     // Si fue obtenido
}
```

Funcionalidades

Interacciones del Usuario

1. Navegación de Objetivos

Función: toggleObjetivo(id)

Comportamiento: Cambia el estado de completado de un objetivo

Efecto visual: Cambia color y estado del chip

2. Gestión de Actividades

Función: iniciarActividad(id)

Comportamiento: Navega a la actividad específica si está disponible

Validación: Verifica disponibilidad antes de navegar

3. Sistema de Favoritos

Función: toggleFavorito(id)

Comportamiento: Marca/desmarca actividades como favoritas

Efecto visual: Cambia color del icono de corazón

4. Cambio de Tema

Función: toggleTheme()

Comportamiento: Alterna entre tema claro y oscuro

Persistencia: Mantiene preferencia durante la sesión

Estados de Componentes

Estados de Evaluación

Completado: Verde con icono de check

Disponible: Azul con icono de play

Bloqueado: Gris con icono de candado

Estados de Progreso

En progreso: Amarillo/naranja

Completado: Verde

Pendiente: Gris

Instalación y Configuración

Prerrequisitos

bashNode.js >= 16.0.0

npm >= 8.0.0

Dependencias Principales

```
json{  
  "vue": "^3.3.0",  
  "vuetify": "^3.4.0",  
  "vue-router": "^4.2.0",
```

```
"@mdi/font": "^7.2.0"  
}
```

Instalación

bash# Clonar el repositorio

```
git clone [repository-url]
```

Instalar dependencias

```
npm install
```

Configurar Vuetify

```
npm install vuetify @mdi/font
```

Ejecutar en desarrollo

```
npm run dev
```

Construir para producción

```
npm run build
```

Configuración de Vuetify

```
javascript// main.js
```

```
import { createApp } from 'vue'
```

```
import { createVuetify } from 'vuetify'
```

```
import 'vuetify/styles'
```

```
import '@mdi/font/css/materialdesignicons.css'
```

```
const vuetify = createVuetify({
```

```
  theme: {
```



```
defaultTheme: 'light',
themes: {
  light: {
    colors: {
      primary: '#667eea',
      secondary: '#764ba2'
    }
  }
}
})
```

app.use(vuetify)

Guía de Uso

Para Estudiantes

1. Navegación Inicial

Comienza leyendo los objetivos de aprendizaje

Revisa tu progreso actual en la sección de progreso

Explora los módulos de contenido en orden

2. Completar Módulos

Expande cada panel de módulo

Lee el contenido detallado

Marca objetivos como completados al finalizar

3. Realizar Actividades

Accede a actividades disponibles

Marca favoritas para acceso rápido

Completa evaluaciones para obtener logros

4. Seguimiento de Progreso

Monitorea la barra de progreso global

Revisa logros obtenidos

Identifica áreas pendientes

Para Educadores

1. Personalización de Contenido

Modifica módulos según curriculum

Ajusta duraciones estimadas

Actualiza descripciones y contenidos

2. Gestión de Actividades

Configura prerequisites de actividades

Ajusta niveles de dificultad

Añade nuevas actividades

3. Sistema de Evaluación

Define nuevos logros

Configura criterios de completado

Monitor progreso de estudiantes

API y Métodos

Métodos de Componente

toggleObjetivo(id: Number)

Alterna el estado de completado de un objetivo específico.

```
javascriptconst toggleObjetivo = (id) => {  
  const objetivo = objetivos.value.find(obj => obj.id === id)  
  if (objetivo) {  
    objetivo.completado = !objetivo.completado  
  }  
}
```

iniciarActividad(id: Number)

Inicia una actividad navegando a su ruta específica.

```
javascriptconst iniciarActividad = (id) => {  
  const actividad = actividades.value.find(act => act.id === id)  
  if (actividad && actividad.ruta && actividad.disponible) {  
    router.push(actividad.ruta)  
  }  
}
```

toggleFavorito(id: Number)

Marca o desmarca una actividad como favorita.

```
javascriptconst toggleFavorito = (id) => {  
  const actividad = actividades.value.find(act => act.id === id)  
  if (actividad) {  
    actividad.favorito = !actividad.favorito  
  }  
}
```

Computed Properties

userProgress

Calcula el progreso global del usuario basado en módulos completados.

isDark

Determina si el tema actual es oscuro para ajustes de UI.

Métodos Auxiliares

getEvaluacionColor(estado: String)

Retorna el color apropiado según el estado de evaluación.

getEvaluacionIcon(estado: String)

Retorna el icono apropiado según el estado de evaluación.

Estilos y Diseño

Paleta de Colores

Primario: #667eea (Azul violeta)

Secundario: #764ba2 (Púrpura)

Éxito: #4CAF50 (Verde)

Advertencia: #FF9800 (Naranja)

Error: #F44336 (Rojo)

Gradientes

Hero Section: linear-gradient(135deg, #667eea 0%, #764ba2 100%)

Efectos de Interacción

Hover en tarjetas: translateY(-5px) con sombra aumentada

Transiciones: all 0.3s ease para suavidad

Box Shadow: 0 8px 25px rgba(0,0,0,0.15) en hover

Responsividad

Mobile First: Diseño adaptativo desde 320px

Breakpoints: sm, md, lg, xl siguiendo Vuetify

Grid System: 12 columnas con ajustes automáticos

Mantenimiento

Actualizaciones de Contenido

Añadir Nuevo Módulo

```
javascriptmodulos.value.push({  
  id: 4,  
  titulo: 'Nuevo Módulo',  
  descripcion: 'Descripción del módulo',  
  duracion: '2 horas',  
  icono: 'mdi-new-icon',  
  completado: false,  
  contenidos: ['Tema 1', 'Tema 2']  
})
```

```
}}
```

Añadir Nueva Actividad

```
javascriptactividades.value.push({
```

```
  id: 4,
```

```
  titulo: 'Nueva Actividad',
```

```
  descripcion: 'Descripción',
```

```
  tipo: 'practica',
```

```
  duracion: '30 min',
```

```
  dificultad: 3,
```

```
  disponible: true,
```

```
  favorito: false,
```

```
  imagen: 'url-imagen',
```

```
  ruta: '/nueva-ruta'
```

```
}}
```

Optimizaciones Recomendadas

Performance

Lazy loading para imágenes de actividades

Virtualización para listas largas

Code splitting por rutas

SEO

Meta tags dinámicos

Structured data para contenido educativo

URLs semánticas

Accesibilidad

ARIA labels para elementos interactivos

Navegación por teclado

Contraste adecuado en todos los temas

Monitoreo y Analytics

Métricas Sugeridas

Tiempo en cada módulo

Tasa de completado por objetivo

Actividades más populares

Progreso promedio de usuarios

Herramientas Recomendadas

Google Analytics para seguimiento

Hotjar para mapas de calor

Sentry para monitoreo de errores

Extensiones Futuras

Funcionalidades Planeadas

Sistema de Gamificación

Puntos por completar actividades

Badges y reconocimientos

Tabla de clasificación

Colaboración Social

Comentarios en módulos

Foros de discusión

Compartir progreso

Adaptabilidad

Rutas de aprendizaje personalizadas

Recomendaciones basadas en progreso

Dificultad adaptativa

Evaluación Avanzada

Quizzes interactivos

Simulaciones en tiempo real

Proyectos colaborativos

Props

question (Object, required): Objeto con la información de la pregunta
questionNumber (Number, required): Número de la pregunta actual
totalQuestions (Number, required): Total de preguntas en el cuestionario
selectedAnswer (String, optional): Respuesta seleccionada por el usuario

Eventos

select-answer: Emitido cuando el usuario selecciona una respuesta

Características

Usa v-radio-group para las opciones de respuesta

Diseño con tarjetas de Material Design

Encabezado que muestra el progreso (Pregunta X de Y)

2. QuizResults.vue

Propósito: Mostrar los resultados finales del cuestionario con retroalimentación detallada.

Props

score (Number, required): Puntaje obtenido
totalQuestions (Number, required): Total de preguntas
questions (Array, required): Array con todas las preguntas
userAnswers (Array, required): Array con las respuestas del usuario

Eventos

restart-quiz: Emitido cuando el usuario quiere reiniciar el cuestionario

Características

Cálculo automático de porcentaje

Iconos y colores dinámicos según el rendimiento

Panel expandible para revisar cada pregunta

Comparación de respuesta del usuario vs respuesta correcta

Botones para reintentar o volver al inicio

Store (Pinia)

Estado (State)

```
typescript{  
  questions: [],          // Array de preguntas obtenidas de la API  
  currentQuestionIndex: 0, // Índice de la pregunta actual  
  userAnswers: [],        // Array de respuestas del usuario  
  quizCompleted: false,   // Indica si el cuestionario está completado  
  score: 0                // Puntaje final obtenido  
}
```

Getters

getCurrentQuestion: Retorna la pregunta actual basada en el índice

getTotalQuestions: Retorna el total de preguntas

getProgress: Calcula el porcentaje de progreso

canGoNext: Verifica si se puede avanzar a la siguiente pregunta

canGoPrevious: Verifica si se puede retroceder a la pregunta anterior

Acciones (Actions)

fetchQuestions()

Obtiene las preguntas desde la API

Inicializa el array de respuestas del usuario

selectAnswer(answerKey)

Guarda la respuesta seleccionada para la pregunta actual

Parámetro: answerKey (string) - clave de la respuesta (a, b, c, d)

nextQuestion() / previousQuestion()

Navega entre preguntas

Incluye validaciones de límites

goToQuestion(index)

Navega directamente a una pregunta específica

Parámetro: index (number) - índice de la pregunta

submitQuiz()

Calcula el puntaje final

Compara respuestas del usuario con respuestas correctas

Marca el cuestionario como completado

resetQuiz()

Reinicia todos los valores a su estado inicial

Permite comenzar el cuestionario nuevamente

API

Endpoint: /api/questions

Método: GET

Descripción: Retorna el array de preguntas para el cuestionario

Estructura de Respuesta

typescript[

```
{
  id: number,           // ID único de la pregunta
  question: string,     // Texto de la pregunta
  answers: {           // Opciones de respuesta
    a: string,
    b: string,
    c: string,
    d: string
  },
```

```
    correctAnswer: string    // Clave de la respuesta correcta (a, b, c, d)
  }
]
```

Ejemplo de Pregunta

```
json{
  "id": 1,
  "question": "¿Qué es un agente inteligente en inteligencia artificial?",
  "answers": {
    "a": "Un programa que actúa aleatoriamente",
    "b": "Un sistema que percibe",
    "c": "Un robot que solo repite órdenes sin aprender",
    "d": "Recibe su entorno y toma decisiones para alcanzar objetivos"
  },
  "correctAnswer": "d"
}
```

Funcionalidades

1. Navegación del Cuestionario

Navegación lineal: Botones "Anterior" y "Siguiente"

Navegación directa: Botones numerados para ir a cualquier pregunta

Validaciones: No permite avanzar sin seleccionar respuesta

2. Indicadores Visuales

Barra de progreso: Muestra el avance del cuestionario

Botones de estado: Cambian de color según si la pregunta está respondida

Contador: Muestra "Pregunta X de Y"

3. Evaluación y Resultados

Cálculo automático: Compara respuestas con las correctas

Puntaje visual: Iconos y colores según el rendimiento

80-100%: Trofeo verde (Excelente)

60-79%: Medalla amarilla (Bueno)

<60%: Pulgar abajo rojo (Necesita mejorar)

4. Revisión Detallada

Panel expandible: Para cada pregunta

Comparación: Respuesta del usuario vs respuesta correcta

Indicadores: Chips de color para mostrar correcta/incorrecta

5. Persistencia

localStorage: Guarda el progreso automáticamente

Recuperación: Restaura el estado al recargar la página

Flujo de Usuario

1. Inicio

Usuario accede a /questions

Sistema carga preguntas desde la API

Se muestra la primera pregunta

2. Respondiendo

Usuario selecciona una respuesta

Sistema guarda la selección

Usuario navega con botones o números

Barra de progreso se actualiza

3. Finalización

Usuario responde todas las preguntas

Botón "Finalizar" se habilita

Sistema calcula puntaje

Se muestran los resultados

4. Revisión

Usuario ve puntaje y porcentaje

Puede expandir cada pregunta para ver detalles

Opciones: reintentar o volver al inicio

Estructura de Datos

Pregunta (Question)

```
typescriptinterface Question {  
  id: number;  
  question: string;  
  answers: {  
    a: string;  
    b: string;  
    c: string;  
    d: string;  
  };  
  correctAnswer: 'a' | 'b' | 'c' | 'd';  
}
```

Estado del Store

```
typescriptinterface QuestionStore {  
  questions: Question[];  
  currentQuestionIndex: number;  
  userAnswers: (string | null)[];  
  quizCompleted: boolean;  
  score: number;  
}
```

Guía de Uso

Para Desarrolladores

Agregar Nuevas Preguntas

Editar server/api/questions.get.ts

Agregar objeto con la estructura requerida

Especificar correctAnswer correctamente

Modificar Estilos

Componentes usan clases de Vuetify

Estilos personalizados en <style scoped>

Colores definidos por tema de Vuetify

Extender Funcionalidad

Agregar nuevas acciones al store

Crear nuevos getters para lógica adicional

Usar eventos para comunicación entre componentes

Para Usuarios Finales

Navegación

Selecciona respuestas haciendo clic en los radio buttons

Usa "Anterior/Siguiente" para navegación lineal

Haz clic en números (1, 2, 3, 4) para ir directamente a una pregunta

Finalización

El botón "Finalizar" solo se activa cuando todas las preguntas están respondidas

Los resultados muestran tu puntaje y rendimiento

Puedes revisar cada pregunta en detalle

Opciones Adicionales

"Reintentar": Comienza el cuestionario desde cero

"Volver al Inicio": Regresa a la página principal

Posibles Mejoras

Funcionalidades Técnicas

Temporizador: Agregar límite de tiempo por pregunta o total

Categorías: Organizar preguntas por temas

Dificultad: Niveles de dificultad progresiva

Explicaciones: Mostrar explicaciones de respuestas correctas

Estadísticas: Historial de intentos y progreso

Mejoras de UX/UI

Animaciones: Transiciones suaves entre preguntas

Temas: Modo oscuro/claro

Accesibilidad: Soporte para lectores de pantalla

Mobile First: Optimización específica para móviles

Gamificación: Badges, logros, ranking

Funcionalidades Avanzadas

Base de Datos: Persistencia servidor

Autenticación: Usuarios registrados

Administración: Panel para gestionar preguntas

Reportes: Análisis de resultados y estadísticas

Exportación: PDF con resultados

Compartir: Resultados en redes sociales

Optimizaciones Técnicas

Lazy Loading: Carga pregunta por pregunta

Caché: Estrategia de cacheo mejorada

PWA: Aplicación web progresiva

Testing: Tests unitarios y de integración

SEO: Optimización para motores de búsqueda

Consideraciones de Mantenimiento

Actualización de Preguntas

Las preguntas están centralizadas en la API

Fácil agregar, editar o eliminar preguntas

No requiere cambios en el frontend

Monitoreo

Errores se logean en consola

Estado se persiste automáticamente

Validaciones previenen estados inválidos

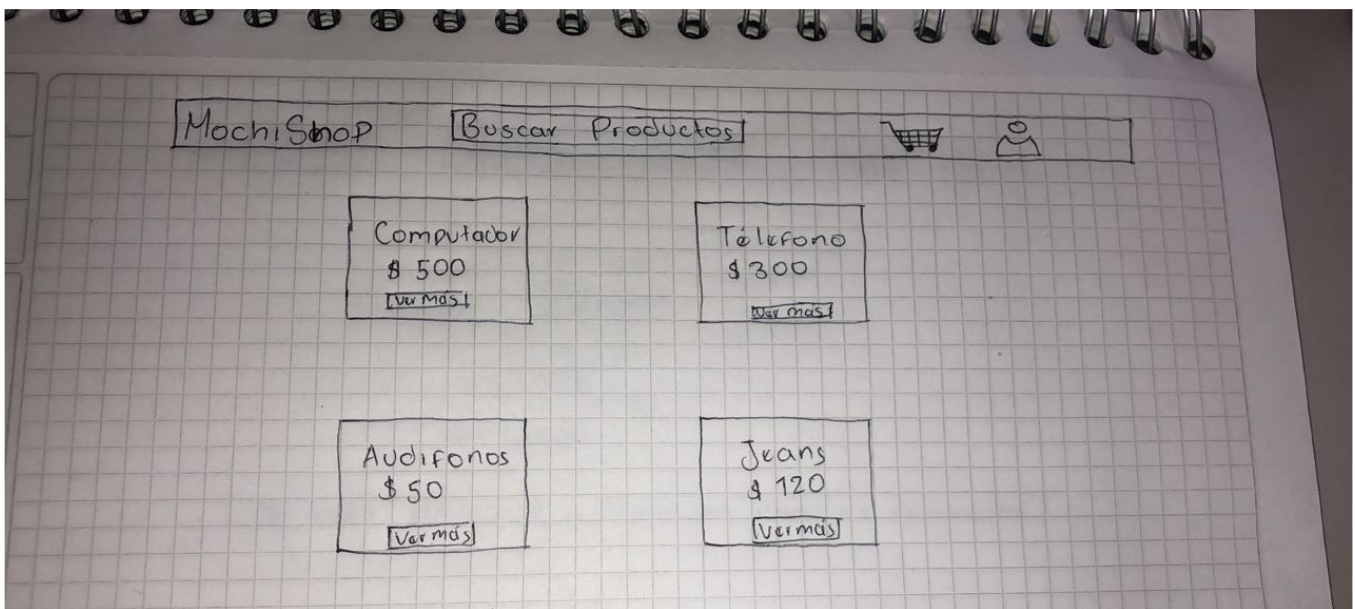
Escalabilidad

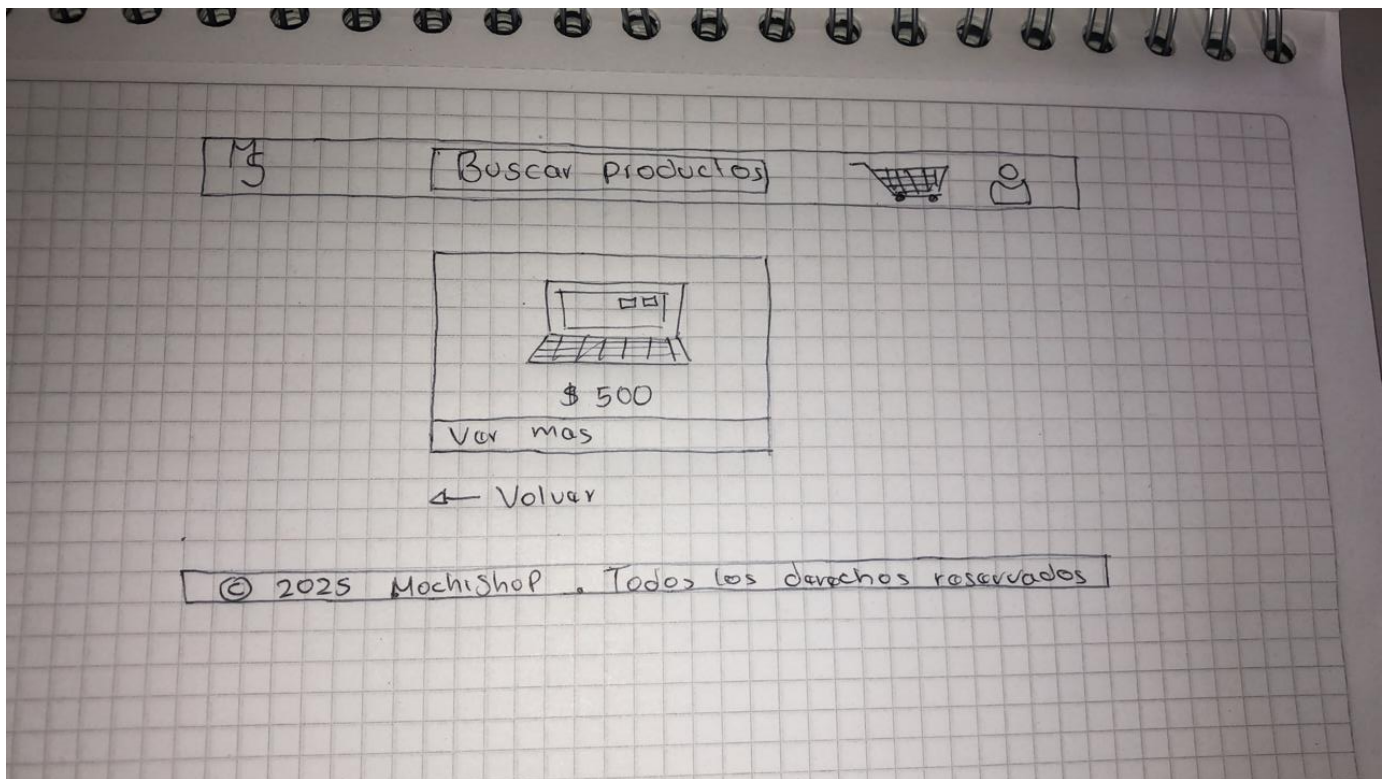
Arquitectura preparada para múltiples cuestionarios

Store modular permite extensión

Componentes reutilizables

MOCKUP Inicial:





- **V - APP – BAR:** v-app-bar Es un componente que se utiliza para crear una barra de navegación superior (o App Bar) en una aplicación. Este componente proporciona una estructura flexible y personalizable para la barra de herramientas de la interfaz, y generalmente se coloca en la parte superior de la aplicación, sirviendo para mostrar el nombre de la aplicación, opciones de navegación y otras herramientas o acciones importantes.

- **BUTTONS:** El componente v-btn es el que se utiliza para crear botones. Este componente es altamente personalizable y permite crear botones de diferentes estilos, tamaños, colores y con distintos tipos de interacciones. Puedes usarlo para acciones en tu interfaz, como enviar formularios, navegar entre vistas, o activar eventos.
- **CARDS:** El componente v-card es una de las estructuras más utilizadas para crear contenedores de contenido estilizados, como tarjetas informativas, presentaciones de productos, perfiles de usuario, entre otros. Las tarjetas proporcionan una forma de mostrar contenido organizado dentro de un bloque con bordes redondeados y sombras.
- **FOOTER COMPONENTS:** El componente v-footer se utiliza para crear un pie de página (footer) que normalmente se coloca al final de la aplicación o de la página. Es una estructura flexible y personalizable que puede contener enlaces, texto, íconos, información de copyright, o cualquier otro contenido que desees mostrar en la parte inferior de la pantalla.
- **AUTOCOMPLETE COMPONENT:** El componente v-autocomplete de Vuetify es un campo de entrada que permite a los usuarios seleccionar un valor de una lista de opciones, mostrando sugerencias mientras escriben. Es especialmente útil cuando tienes una lista de opciones grandes y desees mejorar la experiencia del usuario al permitirles buscar y filtrar elementos a medida que escriben.